YZ 103 · INTRODUCTION TO PROGRAMMING

Making Decisions in Java

Week 4 — boolean logic, if / else, switch, and the traps that come with them.

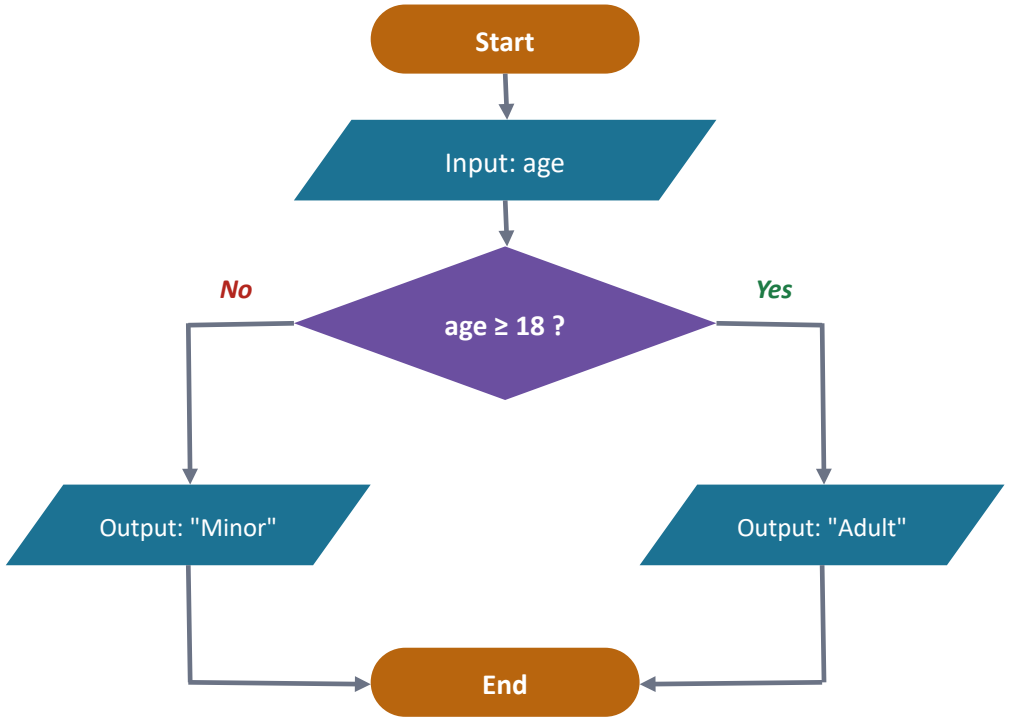
Erciyes University · Dr. Murat Akpulat

THIS WEEK

What We'll Decide Today

1	**boolean and relational operators**	== != < > <= >= — an expression that evaluates to true or false.
2	**if, if-else, else-if**	The decision diamond from Weeks 1–2, in Java.
3	**Braces, indentation, and classic traps**	The stray semicolon, the dangling else.
4	**Logical operators**	&&, ||, ! — truth tables and short-circuit evaluation.
5	**Validation, boundaries, nested conditions**	Validate first, test the edges, then leap year, triangle, BMI.
6	**switch**	Cases, break, fall-through, and the arrow form.
7	**The ternary operator ?:**	A one-line if-else for simple choices.
8	**Comparing Strings and doubles**	Why == is the wrong tool for both.

The Decision Diamond Becomes if / else



Adult.java — inside main

```
Scanner in = new Scanner(System.in);
int age = in.nextInt();
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

purple = the diamond *teal = Input / Output*

Typed input	Program prints
20	Adult
15	Minor

Relational Operators Produce true or false

Op	Meaning	Example	Value
==	equal to	3 == 3	true
!=	not equal to	3 != 4	true
<	less than	2 < 5	true
>	greater than	2 > 5	false
<=	less than or equal	5 <= 5	true
>=	greater than or equal	4 >= 9	false

```
boolean adult = age >= 18;
boolean same = a == b;
```

BOOLEAN

A boolean holds exactly one of two values: true or false. A relational expression IS such a value — you can store it, print it, or test it.

Expressions Are Values

RelDemo.java — inside main

```java
int a = 7, b = 3;
System.out.println(a > b);      // true
System.out.println(a == b);     // false
System.out.println(a != b);     // true
System.out.println(a <= 7);     // true
boolean big = a > 5;
System.out.println(big);        // true
```

OUTPUT

```
true
false
true
true
true
```

Where it shows up

- Inside if (...) — the test.
- In a variable: boolean big = a > 5;
- Printed directly with println.
- Combined with && and || (soon).

= Stores, == Compares

```
int x = 5;
if (x = 5) {              // ✗
    System.out.println("five");
}
```

COMPILER MESSAGE

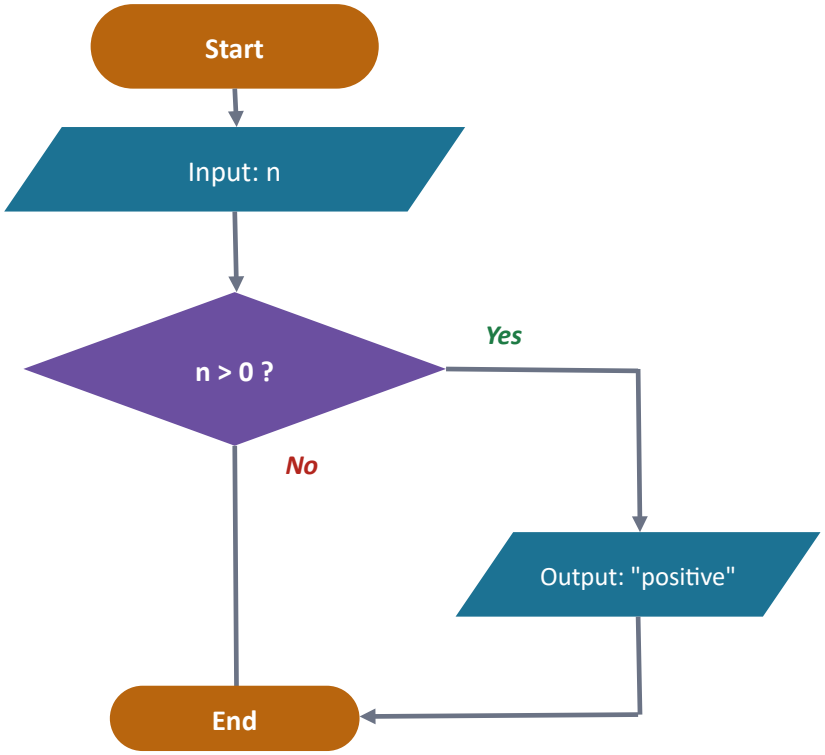
```
ErrAssign.java:4: error: incompatible types:
int cannot be converted to boolean
        if (x = 5) {
               ^
```

The fix: if (x == 5)

ONE CHARACTER, A DIFFERENT PROGRAM

= assigns the right side to the left (the flowchart's ←). == asks a question. Java rejects if (x = 5) because 5 is an int, not a boolean — but if (flag = true) on a boolean variable would compile, and silently overwrite flag.

if: Run a Block Only When the Test Is true



if without else

```
Scanner in = new Scanner(System.in);
int n = in.nextInt();
if (n > 0) {
    System.out.println("positive");
}
// no else: when the test is false, nothing happens
```

1. Java evaluates the condition in ( ).
2. true → run the block { }. false → skip it.
3. Execution continues after the closing brace.

if-else: Exactly One of Two Blocks Runs

EvenOdd.java — inside main

```
Scanner in = new Scanner(System.in);
int n = in.nextInt();
if (n % 2 == 0) {
    System.out.println(n + " is even");
} else {
    System.out.println(n + " is odd");
}
```

Typed input	Program prints
7	7 is odd
10	10 is even

n % 2 == 0 is the even test: the remainder after dividing by 2 is zero.

RULE OF THUMB

if-else is for “this OR that”. When both checks are independent questions, use two separate ifs instead.

Independent ifs vs. else-if

TWO SEPARATE IFS

```
int x = 15;
if (x > 0) {
    System.out.println("Positive");
}
if (x > 10) {
    System.out.println("Greater than 10");
}
```

OUTPUT — both tests ran

```
Positive
Greater than 10
```

ELSE-IF

```
int x = 15;
if (x > 0) {
    System.out.println("Positive");
} else if (x > 10) {
    System.out.println("Greater than 10");
}
```

OUTPUT — the second test was never asked

```
Positive
```

WHICH ONE?

Separate ifs ask every question. else-if asks the next question only if the previous answer was no. Use separate ifs when several things can be true at once.

else-if: A Ladder of Questions

Grade.java — inside main

```
Scanner in = new Scanner(System.in);
int score = in.nextInt();
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
} else if (score >= 60) {
    System.out.println("D");
} else {
    System.out.println("F");
}
```

score	Program prints
95	A
85	B
70	C
59	F
150	A

ORDER MATTERS

The first true test wins and the rest are skipped. Test >= 60 first and a 95 would print D.

Following the Ladder by Hand (score = 85)

Test	Result	What happens
`score >= 90`	`false`	skip this block
`score >= 80`	`true`	print B — then leave the whole chain
`score >= 70`	—	never tested
`score >= 60`	—	never tested
`else`	—	skipped

The same ladder in the wrong order:

```
if (score >= 60) {
    System.out.println("D");
} else if (score >= 70) {
    System.out.println("C");
}   // … and so on
```

With score = 95 this prints D: the first test (>= 60) is already true, so the better grades are never reached.

BOUNDARY CONDITIONS

> or >= ? The Edge Decides

AgeCmp.java — inside main

```
Scanner in = new Scanner(System.in);
int age = in.nextInt();
System.out.println("age > 18  : " + (age > 18));
System.out.println("age >= 18 : " + (age >= 18));
```

age	age > 18	age >= 18
17	false	false
18	false	true
19	true	true

READ THE SENTENCE CAREFULLY

“18 or older” is >= 18. “Older than 18” is > 18. Mixing them up fails for exactly one value — which is why the bug survives casual testing.

Boundary Value Testing

A program accepts ages 18 to 65, inclusive. Which inputs would you test?

17	18	19	64	65	66
reject	*accept*	*accept*	*accept*	*accept*	*reject*

Test each edge, one step below it, and one step above it.

AgeRange.java — inside main

```
if (age >= 18 && age <= 65) {
    System.out.println("accepted");
} else {
    System.out.println("rejected");
}
```

age	program prints
17	rejected
18	accepted
19	accepted
64	accepted
65	accepted
66	rejected

Validate First, Then Decide

WITHOUT VALIDATION

```
if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else if (score >= 70) {
    System.out.println("C");
} else if (score >= 60) {
    System.out.println("D");
} else {
    System.out.println("F");
}
```

TYPED: 150 — a score of 150 earns an A

```
150
A
```

WITH VALIDATION FIRST

```
if (score < 0 || score > 100) {
    System.out.println("Invalid score");
} else if (score >= 90) {
    System.out.println("A");
} else if (score >= 80) {
    System.out.println("B");
} else {
    System.out.println("F");
}
```

score	Program prints
150	Invalid score
-5	Invalid score
85	B

BIG IDEA

A decision structure doesn't only produce answers — it also rejects bad input. Put the invalid-range test FIRST, before the ladder.

CLASSIC TRAPS

Without Braces, if Controls One Statement

Braces.java — inside main

```
int x = 3;
if (x > 10)
    System.out.println("big");
    System.out.println("really big");
```

OUTPUT (x is 3 — the test is false!)

```
really big
```

Indentation is for humans; Java ignores it. Only the first println belongs to the if. The second one always runs.

HOUSE RULE

Always write the braces, even for a single statement. It costs two characters and prevents a whole class of bugs.

CLASSIC TRAPS

The Stray Semicolon

SemiBug.java — inside main

```
int x = 3;
if (x > 10);
{
    System.out.println("x is big");
}
```

OUTPUT (x is 3, and x > 10 is false)

```
x is big
```

if (x > 10); — the semicolon IS the body of the if: an empty statement. The block below it is just a plain block that always runs.

NO ERROR, WRONG ANSWER

This compiles without a single warning. Never put a ; right after if (...).

CLASSIC TRAPS

The Dangling else

Dangling.java — inside main

```
int a = 5, b = 2;
if (a > 3)
    if (b > 3)
        System.out.println("both");
else
    System.out.println("else?");
```

OUTPUT (a = 5, b = 2)

```
else?
```

The else attaches to the NEAREST if — here if (b > 3) — no matter how it is indented.

With braces you decide who owns the else:

```
if (a > 3) {
    if (b > 3) {
        System.out.println("both");
    }
} else {
    System.out.println("else?");
}
```

Now the else belongs to if (a > 3): with a = 5 nothing is printed.

LOGICAL OPERATORS

&&, ||, ! — Truth Tables

AND `a && b`

a	b	a && b
true	true	true
true	false	false
false	true	false
false	false	false

both must be true

OR `a || b`

a	b	a || b
true	true	true
true	false	true
false	true	true
false	false	false

at least one is true

NOT `!a`

a	!a
true	false
false	true

flips the value

READING TIP

&& is the strict one (one false ruins it); || is the forgiving one (one true saves it). The single forms & and | exist but don't short-circuit — stick to && and ||.

Two Conditions, One Decision

Access.java — inside main

```
Scanner in = new Scanner(System.in);
int age = in.nextInt();
boolean hasId = in.nextBoolean();
if (age >= 18 && hasId) {
    System.out.println("Access granted");
} else {
    System.out.println("Access denied");
}
```

age, hasId	Program prints
20 true	Access granted
20 false	Access denied
15 true	Access denied

Run 3 shows why && matters: the person has an ID but is too young, so access is denied.

NEXTBOOLEAN()

Scanner can read a boolean directly: type true or false. Mixed on one line with an int, as above, is fine — Scanner splits on whitespace.

Short-Circuit Evaluation

GUARD FIRST

```
if (x != 0 && 10 / x > 1) {
    System.out.println("ratio above 1");
} else {
    System.out.println("safe");
}
```

x	Program prints
0	safe
2	ratio above 1

GUARD SECOND

```
if (10 / x > 1 && x != 0) {
    System.out.println("ratio above 1");
}
```

x = 0 → RUNTIME ERROR

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Unsafe.main(Unsafe.java:7)
```

WHY

a && b: if a is false, Java never looks at b. a || b: if a is true, b is skipped. So put the check that protects (x != 0) on the left.

Precedence and De Morgan

Prec.java — inside main

```
boolean a = true, b = false, c = false;
System.out.println(a || b && c);     // true
System.out.println((a || b) && c);   // false
System.out.println(!a || b);          // false
System.out.println(!(a && b));        // true
System.out.println(!a || !b);         // true
```

OUTPUT

```
true
false
false
true
true
```

Who goes first?

Order	Operator
1	!
2	&&
3	||

De Morgan's laws

```
!(a && b)  ==  !a || !b
!(a || b)  ==  !a && !b
```

When in doubt, add parentheses — they cost nothing and make intent obvious.

Say It Simply: Condition Clean-Up

Instead of	write
`if (isStudent == true)`	`if (isStudent)`
`if (isStudent == false)`	`if (!isStudent)`
`if (!(age < 18))`	`if (age >= 18)`
`if (a > b) ok = true; else ok = false;`	`boolean ok = a > b;`

Simp.java — inside main

```java
boolean isStudent = false;
int age = 15;
System.out.println(isStudent == false);  // true
System.out.println(!isStudent);          // true
System.out.println(!(age < 18));         // false
System.out.println(age >= 18);           // false
```

OUTPUT

```
true
true
false
false
```

Each pair prints the same value: the short form is the same decision with less noise.

Two Classic Logical Mistakes

1 Math notation

```
if (month == 6 || 7 || 8) {   // ✗
```

COMPILER MESSAGE

```
ErrOr.java:4: error: bad operand types for binary operator '||'
        if (month == 6 || 7 || 8) {
                          ^
```

Each || needs a complete condition on both sides:

```
if (month == 6 || month == 7 || month == 8)
```

2 A condition that can never be true

```
if (x > 10 && x < 5) {
    System.out.println("inside");
}
System.out.println("done");
```

x	program prints
3	done
7	done
12	done

No number is above 10 and below 5 at once. Ask: can both parts be true together? If not, you probably meant ||.

COMPILE ERROR VS. LOGIC ERROR

Mistake 1 is caught at compile time — Java tells you. Mistake 2 compiles and runs, then quietly does nothing useful. The second kind is why we test.

One Combined Condition, or Nested ifs?

COMBINED

Range.java — inside main

```
if (n >= 10 && n <= 50 && n % 5 == 0) {
    System.out.println("valid");
} else {
    System.out.println("invalid");
}
```

n	Program prints
25	valid
7	invalid
55	invalid
22	invalid

NESTED — same behavior

NestedRange.java — inside main

```
if (n >= 10) {
    if (n <= 50) {
        if (n % 5 == 0) {
            System.out.println("valid");
        } else {
            System.out.println("invalid");
        }
    } else {
        System.out.println("invalid");
    }
} else {
    System.out.println("invalid");
}
```

Same four runs, same four outputs. But the nested version needs the “invalid” branch three times; the combined version says it once. Prefer the flat form when every failed test gives the same result; nest when the second question only makes sense after the first.

Leap Year: Mixing && and ||

LeapYear.java — inside main

```
if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {
    System.out.println(year + " is a leap year");
} else {
    System.out.println(year + " is not a leap year");
}
```

A year is a leap year if it is divisible by 4, except century years, which must be divisible by 400.

year	result	why
1900	not a leap year	divisible by 100 but not by 400
2000	a leap year	divisible by 400
2023	not a leap year	not divisible by 4
2024	a leap year	divisible by 4, not by 100

PARENTHESES

(A && B) || C: say what you mean with parentheses even where precedence would work.

Largest of Three Numbers

Max3.java — inside main

```
Scanner in = new Scanner(System.in);
int a = in.nextInt();
int b = in.nextInt();
int c = in.nextInt();
int max = a;
if (b > max) {
    max = b;
}
if (c > max) {
    max = c;
}
System.out.println("max = " + max);
```

a b c	Program prints
4 9 7	max = 9
8 2 8	max = 8

Keep a running best (max) and update it with two separate ifs — the Week 2 “running best” pattern.

SHORTCUT

int max = Math.max(a, Math.max(b, c)); — same result in one line. Writing it out with ifs is the point of this week.

Can These Sides Form a Triangle?

Triangle.java — inside main

```
Scanner in = new Scanner(System.in);
int a = in.nextInt();
int b = in.nextInt();
int c = in.nextInt();
if (a + b > c && a + c > b && b + c > a) {
    System.out.println("valid triangle");
} else {
    System.out.println("not a triangle");
}
```

a b c	Program prints
3 4 5	valid triangle
1 2 5	not a triangle

Triangle inequality

Every pair of sides must add up to MORE than the third. All three tests must hold — hence &&.

BMI Category: Week 3 Plus a Decision

BmiCat.java — inside main

```
Scanner in = new Scanner(System.in);
double weight = in.nextDouble();
double height = in.nextDouble();
double bmi = weight / (height * height);
String category;
if (bmi < 18.5) {
    category = "Underweight";
} else if (bmi < 25) {
    category = "Normal";
} else if (bmi < 30) {
    category = "Overweight";
} else {
    category = "Obese";
}
System.out.printf("BMI: %.1f (%s)%n", bmi, category);
```

kg m	Program prints
70 1.75	BMI: 22.9 (Normal)
95 1.70	BMI: 32.9 (Obese)

ASSIGNED ON EVERY PATH

String category; has no value until a branch sets it. Java checks that EVERY branch does — the else makes that true.

CHOOSING A TOOL

Which Decision Structure Should I Use?

Situation	Use	Example
Two outcomes	`if-else`	adult / minor
Several ranges, one outcome	`else-if ladder`	grade letters
Several independent checks	`separate ifs`	even/odd AND divisible by 5
A second question after the first	`nested if`	user name OK, then password
One variable, many exact values	`switch`	calculator operator
A short either-or value	`?:`	"even" or "odd"

STILL UNSURE?

Start with a plain if-else and refine. The structure should follow the question you are actually asking.

switch: Choosing Among Many Exact Values

Calc.java — inside main

```
Scanner in = new Scanner(System.in);
double a = in.nextDouble();
char op = in.next().charAt(0);
double b = in.nextDouble();
switch (op) {
    case '+':
        System.out.println(a + b);
        break;
    case '-':
        System.out.println(a - b);
        break;
    case '*':
        System.out.println(a * b);
        break;
    case '/':
        System.out.println(a / b);
        break;
    default:
        System.out.println("Unknown operator");
}
```

a op b	Program prints
6 * 7	42.0
7 / 2	3.5
5 ^ 2	Unknown operator

`case x:` jump here when op equals x

`break;` leave the switch

`default:` everything else — like a final else

switch works on int, char, String and enums — not on ranges, doubles, or conditions like x > 5.

Fall-Through: Forgetting break

Fall.java — no break anywhere

```
int n = 2;
switch (n) {
    case 1:
        System.out.println("one");
    case 2:
        System.out.println("two");
    case 3:
        System.out.println("three");
    default:
        System.out.println("other");
}
```

OUTPUT (n = 2)

```
two
three
other
```

Without break, execution keeps running into the next cases until it hits a break or the end.

Used on purpose: grouped cases

```
switch (month) {
    case 4: case 6: case 9: case 11:
        System.out.println(30);
        break;
    case 2:
        System.out.println(28);
        break;
    default:
        System.out.println(31);
}
```

month 4 → 30 month 2 → 28 month 12 → 31

The Arrow Form: No break, No Fall-Through

Arrow.java — inside main

```
Scanner in = new Scanner(System.in);
int day = in.nextInt();
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7 -> "Weekend";
    default -> "Invalid day";
};
System.out.println(type);
```

day	Program prints
3	Weekday
6	Weekend
9	Invalid day

JAVA 14 AND LATER

case a, b -> value; lists several labels, needs no break, never falls through, and switch can produce a value you assign directly. Prefer it when your Java version allows.

TERNARY OPERATOR

?: — a One-Line if-else

```
condition  ?  valueIfTrue  :  valueIfFalse
```

Tern.java — inside main

```
Scanner in = new Scanner(System.in);
int n = in.nextInt();
String parity = (n % 2 == 0) ? "even" : "odd";
int abs = (n < 0) ? -n : n;
System.out.println(n + " is " + parity + ", |n| = " + abs);
```

n	Program prints
-7	-7 is odd, |n| = 7
10	10 is even, |n| = 10

WHEN TO USE IT

For a short either-or value. If a branch needs several statements, or the line stops being readable, write a real if-else.

COMPARING VALUES

Strings: Use equals, Not ==

StringEq.java — inside main

```
String answer = in.next();           // user types: yes
System.out.println(answer == "yes");
System.out.println(answer.equals("yes"));
System.out.println(answer.equalsIgnoreCase("YES"));
```

OUTPUT (typed: yes)

```
false
true
true
```

Why false?

For objects like String, == asks “are these the very same object in memory?”. equals asks “do they contain the same text?”

RULE

Compare text with .equals() (or .equalsIgnoreCase()). Use == for numbers and chars, never for Strings.

doubles: Compare with a Tolerance

DoubleEq.java — inside main

```
double a = 0.1 + 0.2;
System.out.println(a == 0.3);
System.out.println(Math.abs(a - 0.3) < 1e-9);
```

OUTPUT

```
false
true
```

Type	Compare with
int, char	`==`
boolean	`==` (or just the value)
double	`Math.abs(a - b) < tolerance`
String	`.equals(...)`

LINK TO WEEK 3

0.1 + 0.2 is 0.30000000000000004 in binary floating point, so == 0.3 is false. 1e-9 is scientific notation for 0.000000001.

PRACTICE

Your Turn: Three Quick Exercises

1 PREDICT THE OUTPUT

```
int x = 10, y = 20;
if (x > 5) {
    if (y < 15) {
        System.out.println("A");
    } else {
        System.out.println("B");
    }
} else {
    System.out.println("C");
}
```

What is printed?

2 FIND THE BUG

```
int age = 15;
if (age >= 18);
{
    System.out.println("Adult");
}
```

Age is 15. What does it print — and why is that wrong?

3 WRITE THE CONDITION

Write ONE if-condition that is true when n is:

- greater than 10
- less than 100
- divisible by 5

Think first, then check the answers two slides on.

CODE TRACING

Trace It: What Does r End Up As?

TraceEx.java — a = 8, b = 3

```
int a = 8, b = 3, r;
if (a % 2 == 0 && b > 5) {
    r = 1;
} else if (a % 2 == 0 || b > 5) {
    r = 2;
} else {
    r = 3;
}
System.out.println(r);
```

Fill in each row, then say what is printed.

Test	true / false?
`a % 2 == 0`	?
`b > 5`	?
`a % 2 == 0 && b > 5`	?
`a % 2 == 0 || b > 5`	?
`r is set to`	?

METHOD

Work top to bottom, evaluate each test with the real values, and stop at the first branch whose test is true. The others are skipped.

PRACTICE

Answers

1 PREDICT THE OUTPUT

B

x > 5 is true, so the outer if runs. y < 15 is false (y is 20), so the inner else runs: B.

2 FIND THE BUG

Adult (even for age 15)

The ; right after if (age >= 18) is the whole if-body. The braces below are an ordinary block that always runs. Delete the ;.

3 WRITE THE CONDITION

n > 10 && n < 100 && n % 5 == 0

All three parts must hold, so join them with &&. Tested: 25 yes, 100 no, 15 yes, 50 yes, 10 no.

TRACE

r = 2 → prints 2

a % 2 == 0 is true, b > 5 is false, so the && test fails. The || test succeeds, so r = 2.

Three Real Compiler Messages About Decisions

= instead of ==

```
int x = 5;
if (x = 5) { ... }
```

COMPILER MESSAGE

```
ErrAssign.java:4: error: incompatible types: int cannot be converted to boolean
        if (x = 5) {
              ^
```

Two statements, no braces

```
if (x > 3)
    println("big");
    println("really big");
else println("small");
```

COMPILER MESSAGE

```
ErrElse.java:7: error: 'else' without 'if'
        else
        ^
```

Variable used outside its block

```
if (true) {
    int y = 5;
}
System.out.println(y);
```

COMPILER MESSAGE

```
ErrScope.java:6: error: cannot find symbol
        System.out.println(y);
                           ^
```

PUTTING IT TOGETHER

The Week 2 ATM Flowchart, in Java

Atm.java — inside main

```
Scanner in = new Scanner(System.in);
int amount = in.nextInt();
int balance = in.nextInt();
int limit = in.nextInt();
if (amount > limit) {
    System.out.println("Exceeds daily limit");
} else if (amount > balance) {
    System.out.println("Insufficient funds");
} else {
    balance = balance - amount;
    System.out.println("Dispensed " + amount + ", balance " + balance);
}
```

amount balance limit	Program prints
500 1200 1000	Dispensed 500, balance 700
1500 3000 1000	Exceeds daily limit
900 400 1000	Insufficient funds
-50 1200 1000	Dispensed -50, balance 1250

SAME SHAPE — AND A HOLE

Two rejections and one success. But look at the last run: a negative amount sails through. Fixed on the next slide.

A Safer ATM: Reject Bad Amounts First

AtmVal.java — inside main

```
Scanner in = new Scanner(System.in);
int amount = in.nextInt();
int balance = in.nextInt();
int limit = in.nextInt();
if (amount <= 0) {
    System.out.println("Invalid amount");
} else if (amount > limit) {
    System.out.println("Exceeds daily limit");
} else if (amount > balance) {
    System.out.println("Insufficient funds");
} else {
    balance = balance - amount;
    System.out.println("Dispensed " + amount + ", balance " + balance);
}
```

amount balance limit	Program prints
-50 1200 1000	Invalid amount
0 1200 1000	Invalid amount
500 1200 1000	Dispensed 500, balance 700
1500 3000 1000	Exceeds daily limit

THE OLD ATM TRUSTED ITS INPUT

With amount -50 the Week 2 version printed:
“Dispensed -50, balance 1250” — the balance went UP.

WRAP-UP

Summary & What's Next

- ✓ Relational operators produce a boolean; = stores, == compares.
- ✓ if / else if / else: the first true test wins. Always write the braces; never put a ; after if ( ).
- ✓ && and || short-circuit — put the protective test first. ! binds tightest, then &&, then ||.
- ✓ switch picks among exact values; break stops fall-through; the arrow form avoids the trap.
- ✓ Validate input first, test the boundaries (17, 18, 19), and write the simplest condition that is correct.
- ✓ ?: is a one-line if-else; compare Strings with equals and doubles with a tolerance — never with ==.

Next: while, for, and do-while — the loop-back arrow from Weeks 1–2, in Java.